RICE UNIVERSITY

Triton, and One Training Step End to End

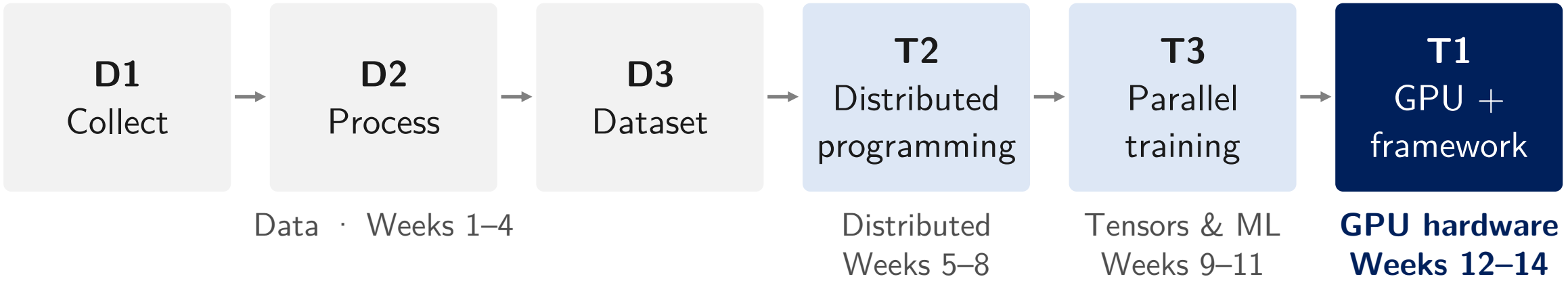
Programs instead of threads, tiles and fusion in Triton, and the whole course in one step

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 14, Lecture 2

Where we are



In our pipeline

Last time: **bits and tiles**. Today: **Triton** writes the tiles for you, then the whole course in **one training step**. T1 is due Friday.

Why do we need Triton?

Last lecture's fast matmul needs:

shared-memory tiles · barriers · register tiles · tensor-core instructions · overlapped loads

Who writes all of that?

CUDA: you, thread by thread
(NVIDIA ships CUTLASS, a template library, to help)

Triton: you write the tile and the loop
the compiler writes the rest

Key idea

Triton's tutorial matmul is a few dozen lines and runs **on par with cuBLAS**. You choose **what** each block computes; the compiler decides **how**.

Source: Triton tutorial "Matrix Multiplication" (triton-lang.org); NVIDIA CUTLASS documentation.

Triton, in plain words

YOU WRITE: ONE PROGRAM

```
@triton.jit
def kernel(x_ptr, ..., BLOCK: tl.constexpr):
    pid = tl.program_id(0)    # which block
    ...                       # block operations
```

A **program** handles **one block** of data; its variables are whole blocks.

→ **Triton compiler**
splits the block over
warps and threads

↓

GPU: a thread block per program
shared memory, registers, tensor cores

Key idea

Programs, not threads: you write what **one block** computes.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

Where Triton shows up

- **torch.compile** writes its fused GPU kernels **in Triton**.
- Triton's tutorials include **fused attention** (FlashAttention style).
- **Lab 3** and **T1**: your own Triton kernel, tested and timed.

The price

Less control than CUDA

First call **compiles**

Autotuning runs **extra** trials

In our pipeline

Why Triton? It writes last lecture's tiling for you, from a short block program. T1 asks for one such kernel.

Source: PyTorch docs, torch.compile and TorchInductor; Ansel et al., "PyTorch 2," ASPLOS 2024. Dao et al., NeurIPS 2022.

Learning goals for today

- Write a Triton kernel with `tl.program_id`, `tl.arange`, **masks**, `tl.load` / `tl.store`, and reductions.
- Write a **tiled matmul** with `tl.dot`, say what the **compiler** does, and **autotune** block sizes.
- Predict what **kernel fusion** saves, as in a fused softmax.
- Follow **one training step** from corpus to cluster to SM.

A puzzle to start

Eager: 5 kernels
max, subtract, exp, sum, divide

Triton: 1 kernel
the same arithmetic

Row softmax of an 8,192 × 1,000 FP32 matrix. Same FLOPs: how much faster can one kernel be?

Key idea

About **4×**, predicted from **bytes alone**: ≈ 262 MB versus ≈ 66 MB of HBM traffic. Softmax is **memory-bound**.

Source: Triton tutorial "Fused Softmax" (traffic count); H100 datasheet (SXM5), 3.35 TB/s.

Part 1

Programming blocks in Triton

What if you wrote the code for a whole block at once?

CUDA's thread view, Triton's block view

	CUDA	Triton
You write code for	one **thread**	one **program** (a block)
Index	a scalar i	a vector of offsets
Shared memory, barriers	by hand	the compiler
Grid, masks, what a block does	you	**you**

Key idea

You decide **what each block computes**; Triton decides how the block's threads do it.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

A masked vector kernel in Triton

```
import triton
import triton.language as tl

@triton.jit
def axpy_kernel(x_ptr, y_ptr, z_ptr, a, n,
                BLOCK: tl.constexpr):
    pid = tl.program_id(axis=0)
    offs = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offs < n
    x = tl.load(x_ptr + offs, mask=mask)
    y = tl.load(y_ptr + offs, mask=mask)
    tl.store(z_ptr + offs, a * x + y, mask=mask)
```

- `pid` = blockIdx.x
- `offs`: the global index for all BLOCK lanes at once
- `mask`: lanes inside the array
- masked lanes are **not** read or written

Key idea

Week 12's $i = b \cdot B + t$, written for a **whole block** as a vector.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

Launching it from PyTorch

```
def axpy(a, x, y, BLOCK=1024):
    assert x.is_cuda and x.is_contiguous()
    assert y.is_contiguous()
    assert y.shape == x.shape
    z = torch.empty_like(x)
    n = x.numel()
    grid = lambda meta: (
        triton.cdiv(n, meta["BLOCK"]),)
    axpy_kernel[grid](x, y, z, a, n,
                      BLOCK=BLOCK)
    return z
```

- grid: programs per axis
- `cdiv`: ceiling division
- tensors pass as pointers
- first call **compiles**; later calls reuse it

Watch out

The first call includes **compilation**: never time it as if it were the kernel.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

Masks handle the tail block

program 3: offs $= 3 \cdot 256 + \text{arange}(256) = 768 \ldots 1023$ (each cell = 8 lanes)



mask true: 768 … 999 (232 lanes) **false: 24 lanes**

- Masked **load**: no memory access; the lane gets `other` (undefined if omitted).
- Masked **store**: nothing is written.

Watch out

For elementwise kernels the masked values are never stored. For **reductions** they would be summed: choose `other` with care.

BLOCK is a compile-time power of two

- `tl.constexpr`: known at compile time $\rightarrow$ each new value **compiles a new kernel**.
- `tl.arange(0, BLOCK)` needs a **power of two**: round up and mask.
- `num_warps` (default 4): BLOCK $= 1024 \rightarrow$ 8 elements per thread.

$$\text{our vocabulary: } N = 322$$
$$\text{next power of two} \geq 322: \;\; 512$$
$$512 - 322 = 190 \text{ masked lanes} \approx 37\%$$

Key idea

Power-of-two rounding wastes **less than half** of the lanes of each row's block.

Source: Triton documentation: `tl.arange`, `triton.next_power_of_2`; course model (vocab 322).

Reductions inside a block

```
x = tl.load(p + offs, mask=mask, other=0.0)
s = tl.sum(x, axis=0)        # sum of the block

x = tl.load(p + offs, mask=mask,
            other=-float("inf"))
m = tl.max(x, axis=0)        # max of the block
```

$$\text{sum:}\quad x + 0 = x$$

$$\text{max:}\quad \max(x, -\infty) = x$$

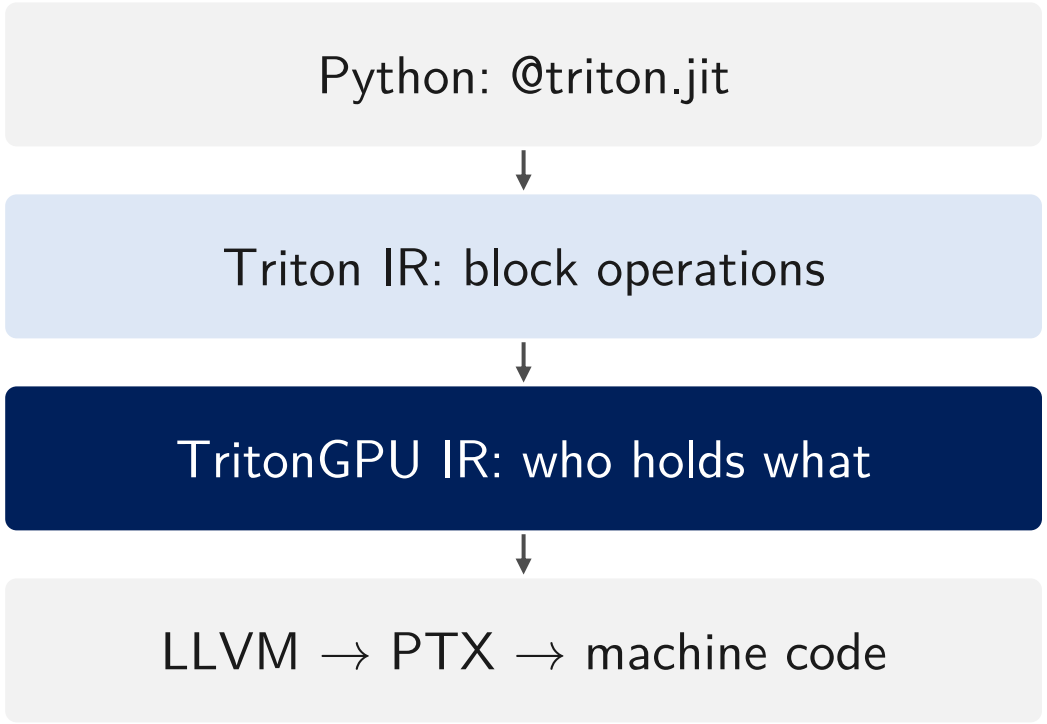
$$\text{product:}\quad x \cdot 1 = x$$

Watch out

Masked lanes must hold the **identity** of the reduction. Forgetting `other` breaks only sizes that are **not** multiples of BLOCK.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

What the compiler does



- Split each block over **warps and threads**
- **Coalesced**, vectorized loads
- `tl.dot` tiles staged in **shared memory**
- `tl.dot` → **tensor-core** instructions
- Load the **next** tile during this one

Key idea

Every decision of last lecture's anatomy, made **by the compiler** from your block code.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

Check yourself

Two minutes with a neighbour.

- n $= 3000$, BLOCK $= 1024$: how many programs? How many lanes are masked?
- A row-max kernel loads with `other=0.0`. For which inputs is it wrong?
- Name two things **you** decide in a Triton kernel, and two the **compiler** decides.

Part 2

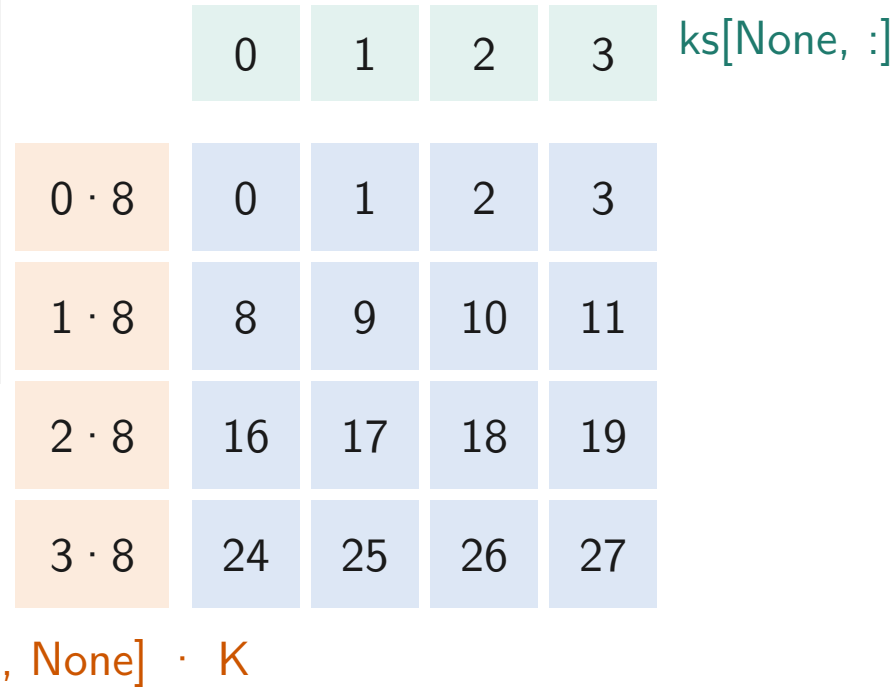
Tiles, softmax, and autotuning

How do last lecture's tiles look in Triton?

2-D blocks by broadcasting

```
i  = pid_m * BM + tl.arange(0, BM)
ks = tl.arange(0, BK)
pa = a + i[:, None] * K + ks[None, :]
A  = tl.load(pa)            # a BM x BK tile
```

example: BM = BK = 4, K = 8



Key idea

Column + row **broadcasts** to a tile of addresses. Each row is contiguous: **coalesced**.

Source: Tillet, Kung & Cox, "Triton," MAPL 2019; Triton documentation and tutorials (triton-lang.org).

A tiled matmul in Triton

```
@triton.jit
def mm(a, b, c, M, N, K, BM: tl.constexpr,
       BN: tl.constexpr, BK: tl.constexpr):
    i = tl.program_id(0) * BM + tl.arange(0, BM)
    j = tl.program_id(1) * BN + tl.arange(0, BN)
    ks = tl.arange(0, BK)
    pa = a + i[:, None] * K + ks[None, :]
    pb = b + ks[:, None] * N + j[None, :]
    acc = tl.zeros((BM, BN), dtype=tl.float32)
    for k in range(0, K, BK):
        acc += tl.dot(tl.load(pa), tl.load(pb))
        pa += BK
        pb += BK * N
    tl.store(c + i[:, None]*N + j[None, :], acc)
```

- 2-D grid: a program per **C tile**
- **FP32** accumulator
- `tl.dot`: one tile product
- advance along **K**

Key idea

Last lecture's loop, in **14 lines**.

Source: Triton tutorial "Matrix Multiplication" (triton-lang.org); Triton API reference (tl.dot, triton.autotune).

Where last lecture's tiles went

Last lecture's level	In the Triton kernel	Who
Block tile in shared memory	`tl.load(pa)`, `tl.load(pb)`	compiler
Register tile, FP32 sums	`acc = tl.zeros(...)`	compiler
Tensor-core instruction	`tl.dot`	compiler
Tile sizes, warps, stages	BM, BN, BK, `num_warps`	**you**

Key idea

The compiler builds the anatomy; **you** pick the tile sizes. Which sizes? **Measure** them.

Source: Triton tutorial "Matrix Multiplication" (triton-lang.org); Triton API reference (tl.dot, triton.autotune).

Softmax overflows unless you subtract the max

$$\text{softmax}(x)[j] = \exp(x[j]) \;/\; \Sigma\, \exp(x[k])$$

$$= \exp(x[j] - m) \;/\; \Sigma\, \exp(x[k] - m), \quad m = \max x$$

$$\text{FP32 largest} \approx 3.4 \times 10^{38} \approx \exp(88.7)$$

Naive, x = [1000, 1000]
$\exp(1000) = \infty \;\rightarrow\; \infty / \infty =$ **NaN**

Stable, m = 1000
$\exp(0) / (1 + 1) =$ **0.5** each

Same value, but every exponent is ≤ 0 and the denominator is ≥ 1.

A fused softmax: one program per row

```
@triton.jit
def softmax_kernel(out_ptr, in_ptr, stride, N,
                   BLOCK: tl.constexpr):
    row = tl.program_id(axis=0)
    cols = tl.arange(0, BLOCK)
    mask = cols < N
    x = tl.load(in_ptr + row * stride + cols,
                mask=mask, other=-float("inf"))
    x = x - tl.max(x, axis=0)
    num = tl.exp(x)
    y = num / tl.sum(num, axis=0)
    tl.store(out_ptr + row * stride + cols, y,
             mask=mask)
```

- grid $=$ (M,): a program per row
- BLOCK: power of two $\geq$ N
- masked lanes: $\exp(-\infty) = 0$
- the row stays **in registers**

Key idea

One read, one write per element: **fused**.

Source: Triton tutorial "Fused Softmax" (triton-lang.org).

Predicting softmax's memory traffic

FP32, M = 8192, N = 1000	Elements moved	Bytes	Q / β
Unfused: 5 PyTorch ops	8MN + 4M	262 MB	≈ 78 µs
Fused: one Triton kernel	2MN	65.5 MB	≈ 19.6 µs

$$I \approx 5\text{MN} \,/\, 8\text{MN} \approx 0.63 \text{ FLOP/byte} \ll 20 \Rightarrow \text{memory-bound}$$

$$\text{predicted speedup} \approx \text{traffic ratio} \approx 4\times$$

Prediction

Each intermediate (33 MB) fits in the 50 MB L2, so expect **less** than 4× — and explain the gap.

Source: Triton tutorial "Fused Softmax" (traffic count); H100 datasheet (SXM5), 3.35 TB/s.

Autotuning: let measurements pick the tiles

```
cfgs = [triton.Config(dict(BM=bm, BN=bn, BK=32),
                      num_warps=w, num_stages=3)
        for bm, bn, w in [(128, 128, 8),
                          (64, 128, 4),
                          (64, 64, 4)]]

@triton.autotune(cfgs, key=["M", "N", "K"])
@triton.jit
def mm(a, b, c, M, N, K, BM: tl.constexpr, ...

grid = lambda m: (triton.cdiv(M, m["BM"]),
                  triton.cdiv(N, m["BN"]))
mm[grid](a, b, c, M, N, K)   # no BM, BN, BK
```

- **Config**: one candidate
- **key**: new shape $\rightarrow$ new search
- first call: **time every** config
- then: the **cached** winner

Key idea

Predict the **range**, measure the **choice**.

Source: Triton tutorial "Matrix Multiplication" (triton-lang.org); Triton API reference (tl.dot, triton.autotune).

Check yourself

Two minutes with a neighbour.

- $M = N = 4096$, $BM = BN = 128$: how many programs does `mm` launch?
- Why must each block dimension of `tl.dot` be at least 16?
- With autotuning, why is the first call for a new shape slow?

Part 3

Fusion, and one step end to end

What does one training step touch, from corpus to SM?

Fusion: fewer bytes, same FLOPs

Unfused: 3 kernels

$t_1 = a \cdot x$ read x, write t_1

$t_2 = t_1 + b$ read t_1, write t_2

$y = \tanh(t_2)$ read t_2, write y

Q = 24n bytes

Fused: 1 kernel

$y = \tanh(a \cdot x + b)$

read x once, write y once

t_1, t_2 stay in registers

Q = 8n bytes

$$\text{same } F; \quad Q\text{: } 24n \to 8n; \quad \text{memory-bound} \Rightarrow \text{predicted speedup} \approx 3\times$$

In our pipeline

T1: write one Triton **reduction or fused elementwise** kernel, and predict its speedup **before** timing it.

What torch.compile fuses

Code	Fused by torch.compile?
A chain of elementwise operations	**yes**: one Triton kernel
A large matrix multiply	usually a library call (cuBLAS)
Code on both sides of a graph break	no: two separate graphs

Key idea

The compiler fuses the **memory-bound glue** between matrix multiplies, and writes it **in Triton**.

Source: PyTorch docs, torch.compile and TorchInductor; Ansel et al., "PyTorch 2," ASPLOS 2024.

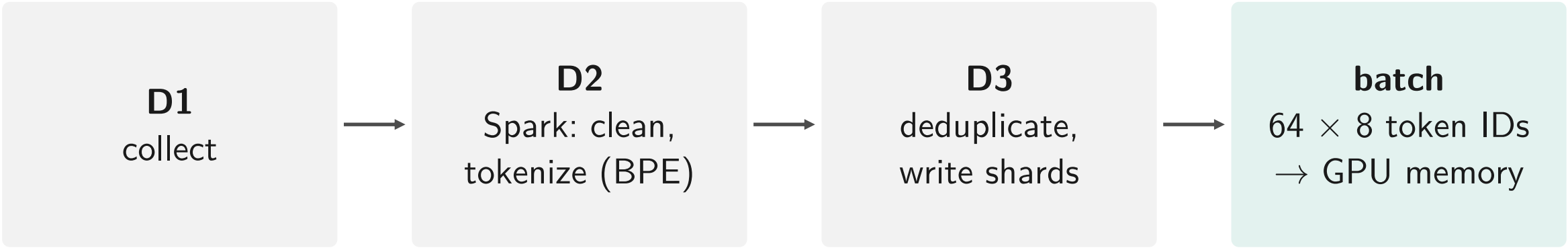
One training step, end to end

Corpus	Cluster	SM
collect, clean tokenize (BPE) deduplicate $\rightarrow$ shards of IDs	split the work over many GPUs all-reduce the gradients	kernels: warps, tiles, tensor cores, fused softmax and loss
Weeks 1–4 · D1–D3	Weeks 5–11 · T2, T3	Weeks 12–14 · T1

Key idea

Every system in the course makes **one** of these zones fast. A step is only as fast as its **slowest** zone.

Corpus: where the batch comes from



- The **tokenizer** sets how many tokens, so how many steps.
- **Duplicates** waste steps on text the model has already seen.

In our pipeline

The fastest step is one you **do not need**: a clean, deduplicated corpus.

Cluster: one step on many GPUs

Problem		System	Where
Sum the gradients	→	**all-reduce: ring, tree; NCCL**	Wk 6–8
Every GPU holds the model	→	**ZeRO, FSDP: shard the state**	Wk 9
One layer is too big	→	**tensor parallelism = 2D matmul**	Wk 10–11
The model is too deep	→	**pipeline stages**	Wk 10

Key idea

Each answer trades **communication** for **memory**; $\alpha + \beta n$ priced every one of them.

SM: one matrix multiply inside one GPU

Launch a kernel; Python runs ahead	Wk 12
Blocks on SMs, warps of 32 hide latency	Wk 12
$h \cdot W_2$: tiles, BF16 tensor cores, FP32 sums	Wk 14
softmax + loss: one fused Triton kernel	today
Roofline: predict each kernel's time	Wk 13

Watch out

At batch 64 our MLP is **launch-bound** (Week 13): fuse, compile, or batch more.

Three ideas that kept coming back

Idea	Data · Wk 1–4	Cluster · Wk 5–11	GPU · Wk 12–14
Count first	shuffle bytes	$\alpha + \beta n$; 16 B/param	FLOPs, bytes, roofline
Move fewer bytes	dedup; combine early	sharding; reduce-scatter	tiles, fusion, fewer bits
Keep workers busy	avoid skew	overlap; small bubbles	many warps; async

Key idea

A system's speed is set by **what it moves** and **how well it overlaps** — and you can **predict** it before you run it.

Check yourself

Two minutes with a neighbour.

- Three elementwise kernels are fused into one. Passes before and after? Speedup?
- Corpus, cluster, or SM: an all-reduce, a `tl.dot`, MinHash deduplication?
- What limits our MLP's step at batch 64, and what would help?

IN-CLASS BUILD

Blocks, masks, and reductions

Write it, test it, time it. Work in pairs.

Exercise: a row-sum kernel

```
@triton.jit
def rowsum_kernel(x_ptr, out_ptr, N, stride,
                  BLOCK: tl.constexpr):
    row = tl.program_id(axis=0)
    cols = tl.arange(0, BLOCK)
    x = tl.load(x_ptr + row * stride + cols,
                mask=_____, other=_____)
    tl.store(out_ptr + row, _____)
```

TASKS

1. Fill the three blanks.
2. Grid and BLOCK for M $\times$ N $=$ 8192 $\times$ 1000.
3. Predict Q and the time (FP32).
4. List five test shapes.

USE

$\pi = 67$ TFLOP/s, $\beta = 3.35$ TB/s, ridge ≈ 20 FLOP/byte. Count one add per element.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

Exercise: solutions

```
    x = tl.load(x_ptr + row * stride + cols,
                mask=cols < N, other=0.0)
    tl.store(out_ptr + row, tl.sum(x, axis=0))
```

Question	Answer
Grid, BLOCK	(8192,); `next_power_of_2(1000)` $= 1024$
Q, time	$4MN + 4M \approx 32.8$ MB $\rightarrow$ ≈ 9.8 µs ($I \approx 0.25$)
Shapes	(1, 1); N $= 1023, 1024, 1025$; M $= 0$; x[:, :500]; large

Key idea

Every shape tests one line: the mask, `other`, the stride, the empty grid.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

What to test, and how close is close enough

dtype	rtol	atol	machine ε
float32	1.3×10^{-6}	10^{-5}	1.2×10^{-7}
bfloat16	1.6×10^{-2}	10^{-5}	7.8×10^{-3}

$$|\text{actual} - \text{expected}| \leq \text{atol} + \text{rtol} \cdot |\text{expected}|$$

- $n = 0$, $n = 1$
- $n = \text{BLOCK} - 1$, BLOCK, $\text{BLOCK} + 1$
- large, non-multiple n
- huge values (overflow)
- non-contiguous views

Key idea

Reference in **higher precision** from the same inputs; loosen a tolerance only with a stated reason.

Source: PyTorch documentation, torch.testing.assert_close (default tolerances).

A test you can run

```
SHAPES = [(1, 1), (3, 255), (3, 256),
          (3, 257), (64, 322), (0, 8)]
DTYPES = [torch.float32, torch.bfloat16]

def test_softmax():
    torch.manual_seed(0)
    for (M, N), dt in product(SHAPES, DTYPES):
        x = torch.randn(M, N, device="cuda")
        x = (100 * x).to(dt)   # overflow test
        y = softmax(x)
        ref = torch.softmax(x.float(), dim=-1)
        assert_close(y, ref.to(dt))
```

- edge shapes around BLOCK
- (64, 322): our logits
- (0, 8): empty input
- $\times$ 100: exp would overflow

Watch out

A kernel that passes only power-of-two shapes is **not** tested.

Then time it as in Week 12: synchronize, warm up, report the median.

Source: PyTorch documentation, torch.testing.assert_close (default tolerances). Timing: Week 12, Lecture 2.

Key takeaways

- Triton programs a **block**: `program_id`, `arange`, **masks**, `tl.load` / `tl.store`, `tl.sum`.
- `tl.dot` + an FP32 accumulator = last lecture's **tiled matmul**; the compiler builds the levels.
- **Autotuning** measures the tile sizes; **fusion** cuts bytes for memory-bound chains.
- One step = **corpus** (D1–D3) $\rightarrow$ **cluster** (Weeks 5–11) $\rightarrow$ **SM** (Weeks 12–14).
- Count first, move fewer bytes, keep every worker busy.

Next time, and readings

When	What
Friday, April 23	T1 due
April 28 – May 4	Final exam (finals period; date and room from the registrar)
Tuesday, May 4	Capstone due (no slip days)

READINGS

Tillet, Kung & Cox, "Triton: an intermediate language and compiler for tiled neural network computations," MAPL 2019.

Triton tutorials: "Fused Softmax" and "Matrix Multiplication".